

Trabalho 3: Sistema de Matchmaking para Jogos Online

Pablo Levy Fernades Alcântara

Raul Medici Martinelli

10 de Maio de 2026

1 Implementação dos Algoritmos de Ordenação

Os algoritmos de ordenação foram implementados na classe `Matchmaking` para organizar os jogadores no array estático. O *Insertion Sort* (`sortByScoreInsertion`) utiliza dois laços de repetição; a cada iteração, um jogador é comparado com os antecessores e inserido na posição correta enquanto os valores maiores são deslocados. O *Merge Sort* (`sortByScoreMerge`) utiliza a estratégia de divisão e conquista por meio da função recursiva `mergeSort` e da função auxiliar `merge`, que intercala subarrays usando alocação dinâmica temporária. Em ambos, o critério principal é a ordem crescente de `score` e, em caso de empate, o menor `timestamp` (ordem de chegada).

2 O Uso da Recursão no Merge Sort

A recursão é fundamental para o *Merge Sort*. A função `mergeSort(int left, int right)` realiza chamadas a si mesma para dividir o array repetidamente pela metade até atingir o caso base (tamanho 1). Após essa fragmentação, o sistema executa a volta da recursão, chamando `merge` para intercalar as partes e reconstruir o array de forma ordenada.

3 Formação de Grupos e Decisões de Design

A ordenação prévia permite que jogadores com habilidades semelhantes ocupem índices contíguos. O método `formGroup` utiliza uma janela deslizante de tamanho `groupSize` para verificar se a diferença entre o maior e o menor `score` do bloco é menor ou igual ao `delta`.

Uma decisão de design importante foi fazer com que `formGroup` apenas retorne o ponteiro para o grupo formado, sem exibi-lo no console. Essa separação de responsabilidades permite que o sistema seja flexível: a lógica de exibição fica a cargo da `main`, enquanto o grupo retornado pode ser utilizado para outros fins de processamento sem obrigatoriedade de saída de dados imediata.

4 Análise de Custo e Gerenciamento de Memória

Abaixo, detalhamos a complexidade e observações sobre a memória:

- **Inserção (`insert`):** $\mathcal{O}(1)$. Adiciona o jogador ao final do array usando o índice `size`.
- **Remoção (`removePlayer`):** $\mathcal{O}(n)$. Requer a busca do ID e o deslocamento de elementos para preencher a lacuna.
- **Ordenação (`Insertion Sort`):** $\mathcal{O}(n^2)$. Custo quadrático devido aos deslocamentos sucessivos no pior caso.
- **Ordenação (`Merge Sort`):** $\mathcal{O}(n \log n)$. Eficiência superior através da divisão logarítmica.
- **Formação de grupo:** $\mathcal{O}(n)$. Varredura linear sobre o array ordenado.

Devido ao tamanho da constante `MAX_PLAYERS` (100.000), o objeto `Matchmaking` ocupa um espaço significativo de memória. Para evitar erros de *stack overflow* durante a execução, foi necessário utilizar a palavra-chave `static` ao instanciar a classe na `main`, garantindo que a estrutura seja alocada no segmento de dados e não na `stack`, que possui limites mais restritos.

5 Comparação Experimental e Visualização

Para a análise de performance, foram realizados testes com entradas de 1.000 a 10.000 jogadores. Como funcionalidade extra, o sistema utiliza bibliotecas nativas do C++ para exportar os tempos de execução em um arquivo `.csv`.

Visto que o C++ não possui ferramentas nativas de plotagem, este arquivo é lido por um script auxiliar em Python (`plot.py`), que renderiza um gráfico comparativo. Essa abordagem permite visualizar claramente a disparidade entre o crescimento quadrático do *Insertion Sort* e a eficiência quase linear do *Merge Sort*, validando a análise assintótica teórica.

Nº de Jogadores	Insertion Sort (ms)	Merge Sort (ms)
1000	17	4
2500	231	12
5000	1141	42
7500	2915	58
10000	6451	125

Table 1: Tempos de execução registrados no arquivo `resultados_ordenacao.csv`.

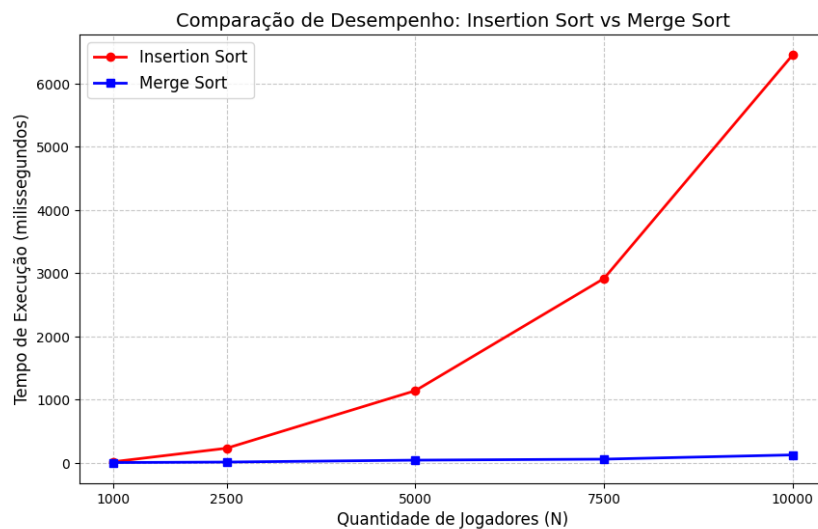


Figure 1: Gráfico de comparação de desempenho entre Insertion Sort e Merge Sort gerado pelo script `plot.py`.